

Temperature vs Top-p

Two settings that control how a language model chooses its next word. Explained simply, with examples.

In Plain English — 3 Sentences

1

Temperature controls how adventurous the model is. At temperature=0 it always picks the safest, most expected word. At temperature=1 it sometimes picks a more surprising word — like the difference between always ordering your usual coffee vs occasionally trying something new off the menu.

2

Top-p works like a bouncer that bans unlikely words before any picking happens. It adds up the most likely words one by one until their combined chance reaches p (say, 90%) — then it blocks every other word from even being considered. The model can only pick from the words that made it past the bouncer.

3

The key difference: temperature changes how the model feels about each word; top-p changes which words are even in the running. Temperature reshapes the whole list of options. Top-p just removes the bottom of that list entirely.

One-line cheat sheet

temperature

How bold should the model be? 0 = always safe. 1 = sometimes surprising.

top-p

Which words are even allowed? 0.9 = only the top 90%-likely words. 1.0 = all words.

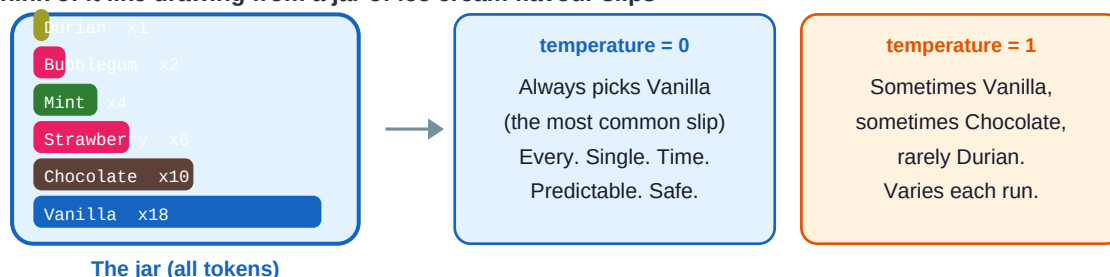
Temperature — Explained with Examples

A simple analogy, then real model outputs at different temperature values.

The Analogy: A jar of ice cream flavour slips

Imagine each possible next word is written on a slip of paper inside a jar. Common words have many slips. Rare words have very few. Temperature controls how you reach in:

Think of it like drawing from a jar of ice cream flavour slips



temperature=0 — You look into the jar and always pick the slip with the most copies. Completely predictable. Run it 10 times, get the same answer 10 times.

temperature=1 — You reach in blindly. You usually grab Vanilla (it has the most slips) but sometimes you grab Chocolate or Strawberry. Very rarely you pull out Durian. Every run can be different.

Real example — same prompt, three temperature values

Prompt: "Write the opening line of a story about a lost dog."

Setting	What the model outputs	Run it again?
temperature = 0 (safe, greedy)	"The little dog had been wandering the streets for three days, his collar jingling softly with every step."	Identical output. Exactly the same words every time.
temperature = 0.7 (balanced)	"Max had not meant to get lost — it happened the moment he chased a butterfly through the gap in the garden fence."	Similar style and tone, but different words each run.
temperature = 1.5 (adventurous)	"Nobody tells you how loud silence is until you are a small dog alone in a city that does not know your name."	Very different each run. Creative, but occasionally odd.

Notice: the story quality is not better or worse at any temperature. It is just different. temperature=0 is best when you need the same reliable answer every time (like code or facts). temperature=1 is best when you want variety and creativity.

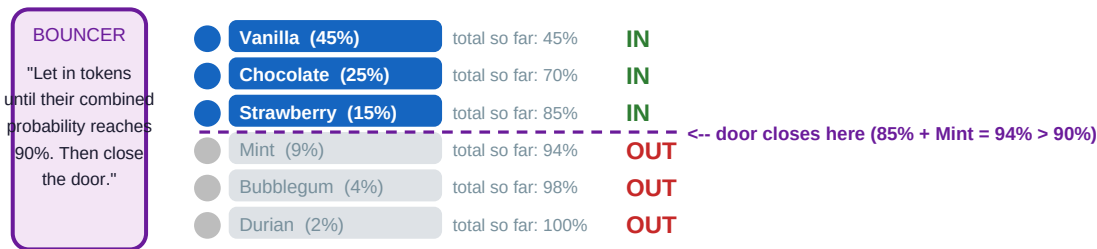
Top-p — Explained with Examples

A simple analogy, then real model outputs at different top-p values.

The Analogy: A bouncer with a capacity rule

Imagine a club (the model's next-word choice). All possible next words queue up outside. The bouncer lets in the most likely words one by one, keeping a running total of their combined probability. The moment the total hits p (e.g., 90%), the door closes. Everyone else is turned away — even if they are perfectly valid words.

Think of it like a bouncer at a club with a capacity rule



top-p=0.9 means: let in words until their combined probability reaches 90%, then stop. In the example above, Vanilla (45%) + Chocolate (25%) + Strawberry (15%) = 85%. Adding Mint would push it to 94% — over budget. So Mint and everything below is banned.

top-p=1.0 means: let everyone in. No filtering. This is the default — top-p does nothing at 1.0.

Real example — same prompt, three top-p values

Prompt: "Suggest one word to describe a sunset."

Setting	Words that can be picked	Sample outputs
top-p = 0.5 (very tight)	Only the top 1-2 most likely words qualify (e.g., 'beautiful' at 48% already fills the 50% budget). Almost always picks the same word.	"Beautiful." "Beautiful." "Beautiful."
top-p = 0.9 (moderate)	The top 5-6 words qualify. Common words like 'breathtaking', 'golden', 'stunning', 'magnificent' are all eligible.	"Golden." "Breathtaking." "Magnificent."

top-p = 1.0 (no filter)	Every word in the vocabulary is eligible, including very rare or unexpected ones like 'ephemeral', 'mercurial', or 'wistful'.	<i>"Wistful." "Golden." "Ephemeral."</i>
--------------------------------	---	--

Notice: top-p does not change how bold the model is — it just cuts out the very unlikely words. Think of it as a safety net, not a creativity dial. Most developers leave top-p at 0.9 or 1.0 and only tune temperature.

The Difference at a Glance + Code

Side-by-side comparison, then simple Python code to try it yourself.

Temperature vs Top-p — Side by Side

	Temperature	Top-p
What it does	Makes the model bolder or safer with every word it considers	Removes the least-likely words from the pool before picking
Analogy	Volume on a radio — turns all stations louder or quieter	Bouncer — some stations simply cannot get in
Default	1.0 (use the raw probabilities as-is)	1.0 (no filtering — all words are allowed)
Set it lower to...	Get more predictable, consistent output	Allow only the most common / expected words
Set it higher to...	Get more varied, creative output	Allow more unusual and surprising words
Use for	Creative writing, brainstorming, factual Q&A;	Avoiding truly random nonsense; rarely needs tuning
Key rule	Tune this one first. It is the main creativity dial.	Leave at 1.0 unless you have a specific reason to change it.

Simple Python Code to Try It Yourself

```
import openai

client = openai.OpenAI() # needs OPENAI_API_KEY set in your environment

def ask(prompt, temperature, top_p):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{'role': 'user', 'content': prompt}],
        temperature=temperature,
        top_p=top_p,
        max_tokens=80,
    )
```

```
return response.choices[0].message.content

prompt = "Write the opening line of a story about a lost dog."

# Safe and consistent (temperature does the work, top_p left at default)
print(ask(prompt, temperature=0, top_p=1.0))

print(ask(prompt, temperature=0, top_p=1.0)) # <- identical to the one above

# Balanced and varied
print(ask(prompt, temperature=0.7, top_p=1.0))

# Creative and surprising
print(ask(prompt, temperature=1.2, top_p=1.0))

# Top-p as a filter (keeps only common words, temperature still adds variety)
print(ask(prompt, temperature=1.0, top_p=0.8))

# Note: OpenAI recommends changing temperature OR top_p -- not both at once.
```

Practical advice: In 90% of cases, just set **temperature** and leave **top_p** at 1.0. Use **temperature=0** for anything that needs to be consistent (code, structured data, factual answers). Use **temperature=0.7** to **1.0** for anything creative. Only reach for **top_p** if you find the model occasionally producing nonsensical or off-topic words.